

CS4423 - Networks

Prof. Götz Pfeiffer

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

1. Graphs and Graph Theory

Lecture 4: Bipartite Graphs and Projections ¶

We'll look at further properties of graphs and networks, both from a theoretical point of views and from the practical side of handling graphs in the `NetworkX` environment. Start by importing the necessary `python` libraries into this `jupyter` notebook.

```
In [1]: import networkx as nx
import numpy as np
```

Also, let's fix some options for drawing graphs.

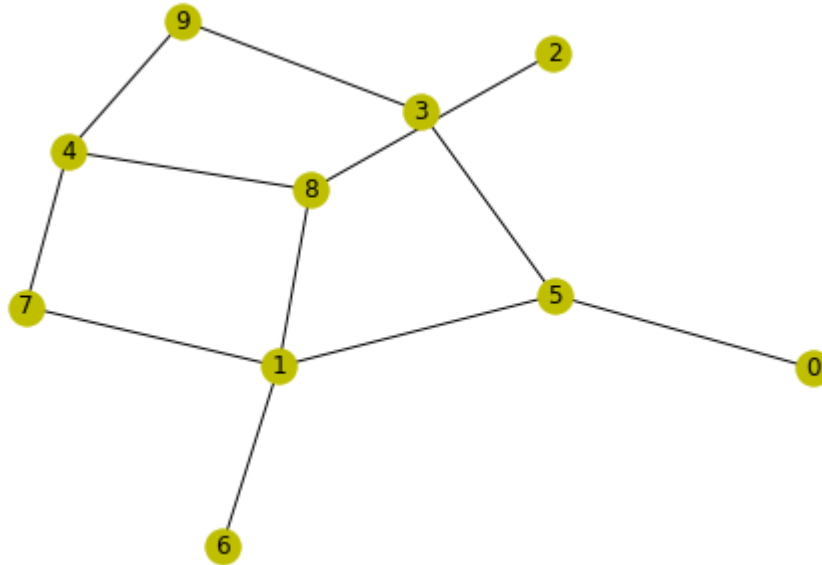
```
In [2]: opts = { "with_labels": True, "node_color": 'y' }
```

Bipartite Graphs

A (simple) graph $G = (X, E)$ is called **bipartite**, if the vertex set X is a disjoint union of two sets B (of black nodes) and W (of white nodes) so that each edge in E links a black vertex with a white vertex.

Here is a sample bipartite graph B , specified to the `Graph` constructor by its edge list.

```
In [3]: edges = [(0,5), (1,5), (1,6), (1,7), (1,8),  
                (2,8), (3,5), (3,9), (4,7), (4,8), (4,9)]  
B = nx.Graph(edges)  
nx.draw(B, **opts)
```



In this graph, the **white** nodes can be taken as the set $W = \{0, 1, 2, \dots, 4\}$ and the **black** nodes as $B = \{5, 6, \dots, 9\}$. The drawing command `nx.draw` takes as optional argument a dictionary `pos` that specifies for each node a (relative) position in the drawing. Here, the node is the key and the position is a pair of x, y -coordinates. In this example we can use the (integer) quotient and remainder, as returned by the python method `divmod` to quickly compute a dictionary of positions that have the white nodes on the left, and the black nodes on the right.

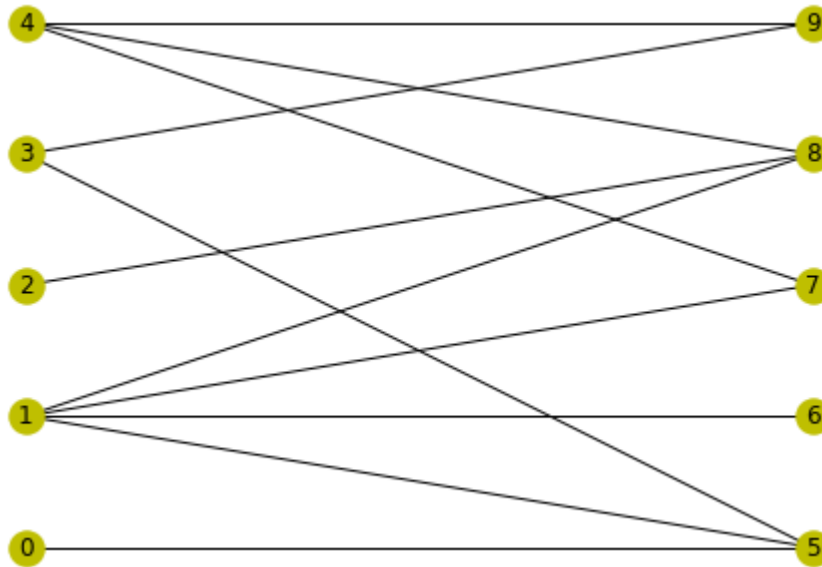
```
In [4]: divmod(7, 5)
```

```
Out[4]: (1, 2)
```

```
In [5]: pos = {x: divmod(x, 5) for x in range(10)}
print(pos)
```

```
{0: (0, 0), 1: (0, 1), 2: (0, 2), 3: (0, 3), 4: (0, 4), 5: (1, 0), 6:
(1, 1), 7: (1, 2), 8: (1, 3), 9: (1, 4)}
```

```
In [6]: nx.draw(B, pos, **opts)
```



Node colors can be specified as a *list* assigned to the keyword argument `node_color`. We can use the *x*-coordinates of the node positions for that purpose.

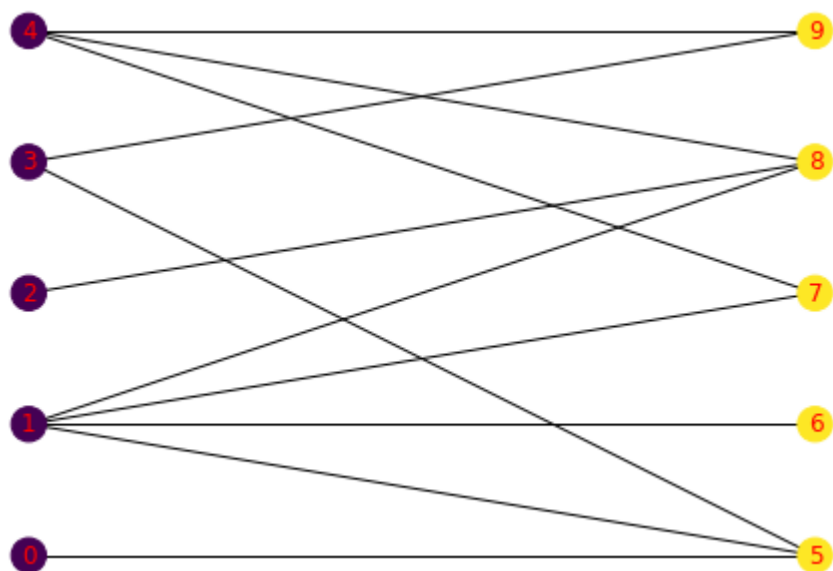
```
In [7]: color = [pos[x][0] for x in B.nodes()]
color
```

```
Out[7]: [0, 1, 0, 1, 1, 1, 0, 0, 1, 0]
```

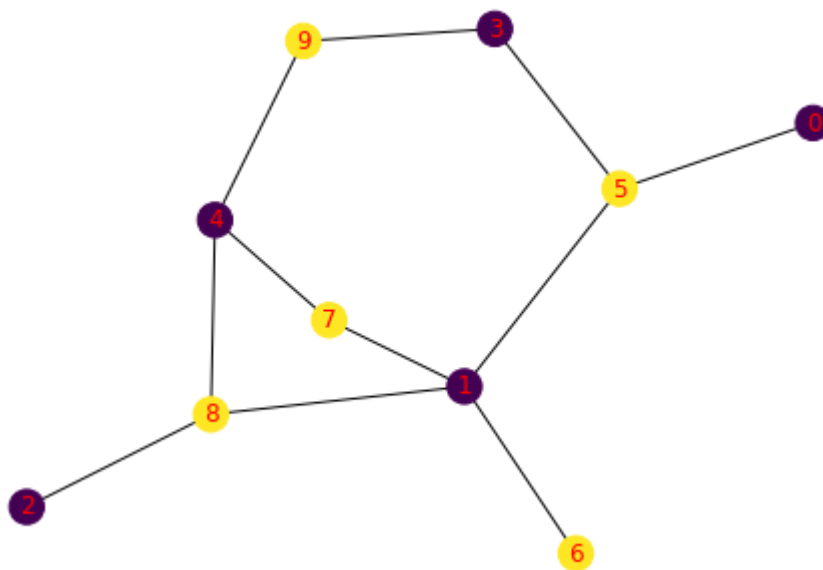
```
In [8]: print(B.nodes)
```

```
[0, 5, 1, 6, 7, 8, 2, 3, 9, 4]
```

```
In [9]: opts2 = { "with_labels": True, "node_color": color, "font_color": 'r' }  
nx.draw(B, pos, **opts2)
```



```
In [10]: nx.draw(B, **opts2)
```



A **(vertex)-coloring** of a graph G is an assignment of (finitely many) colors to the nodes of G , so that any two nodes which are connected by an edge have *different* colors.

A graph is called N -**colorable**, if it has a vertex coloring with (at most) N colors.

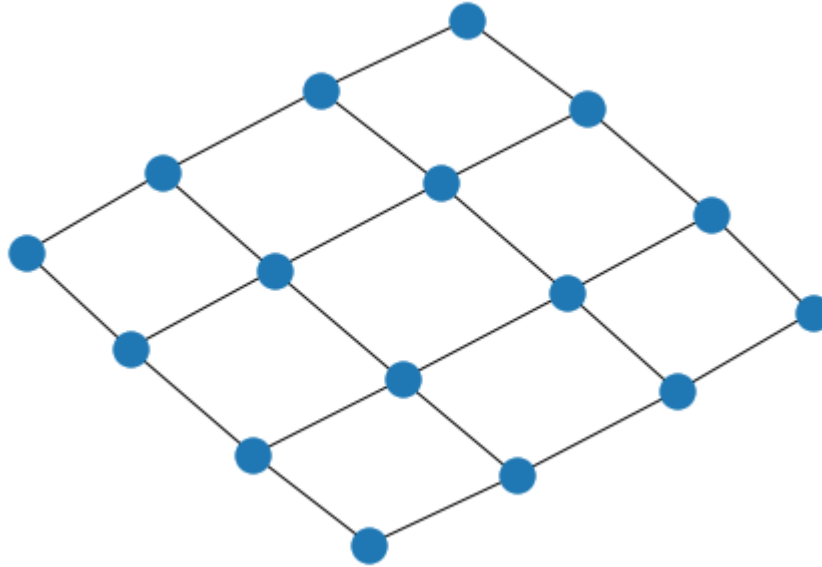
Theorem. Let G be a graph. The following are equivalent:

- G is bipartite;
- G is 2-colorable;
- each cycle in G has even length.

(See below for **cycle** and **length**)

2D grids are naturally bipartite:

```
In [11]: G44 = nx.grid_2d_graph(4, 4)
          nx.draw(G44)
```



The method `nx.bipartite.color` determines a 2-coloring of a graph G algorithmically, if it exists, i.e. if G is bipartite.

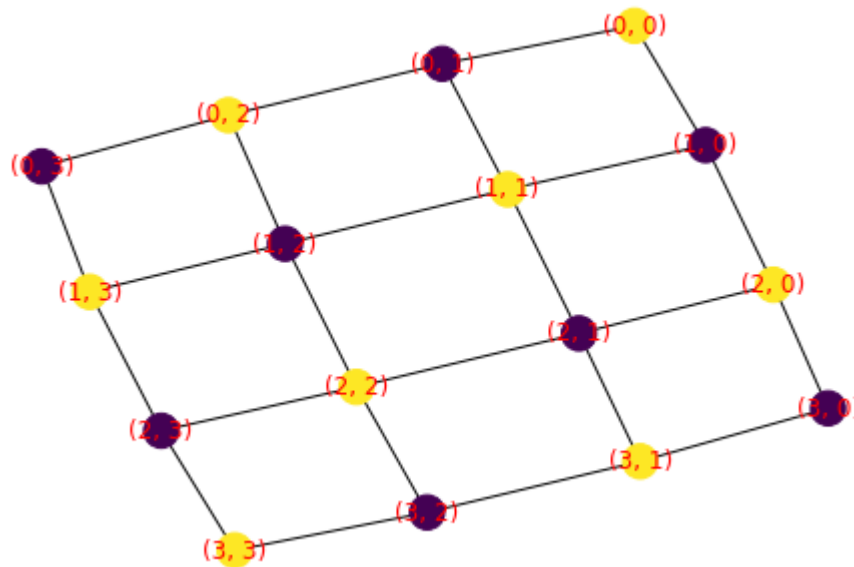
```
In [12]: color = nx.bipartite.color(G44)
          print(color)
```

```
{(0, 0): 1, (1, 0): 0, (0, 1): 0, (1, 1): 1, (0, 2): 1, (1, 2): 0, (0, 3): 0, (1, 3): 1, (2, 3): 0, (3, 3): 1, (2, 2): 1, (3, 2): 0, (2, 1): 0, (3, 1): 1, (2, 0): 1, (3, 0): 0}
```

```
In [13]: color = [color[x] for x in G44.nodes()]
          color
```

```
Out[13]: [1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1]
```

```
In [14]: opts2["node_color"] = color
         nx.draw(G44, **opts2)
```



Affiliation Networks and Projections

Bipartite graphs arise in practice as models for **affiliation networks**. In such a network, the *black* nodes are people, and the *white* nodes are attributes of the people, such as common interests (books bought online, TV shows watched), workplaces, social events attended ... Edges in such network connect people with their attributes.

A frequently cited example from the sociology literature (Davis, A., Gardner, B., and Gardner, R. 1941. Deep South. Chicago: University of Chicago Press.) is the **Southern Women Network**. This is a data set of 18 women observed over a nine-month period. During that period, various subsets of these women met in a series of 14 informal social events. The following table records which women attended which events.

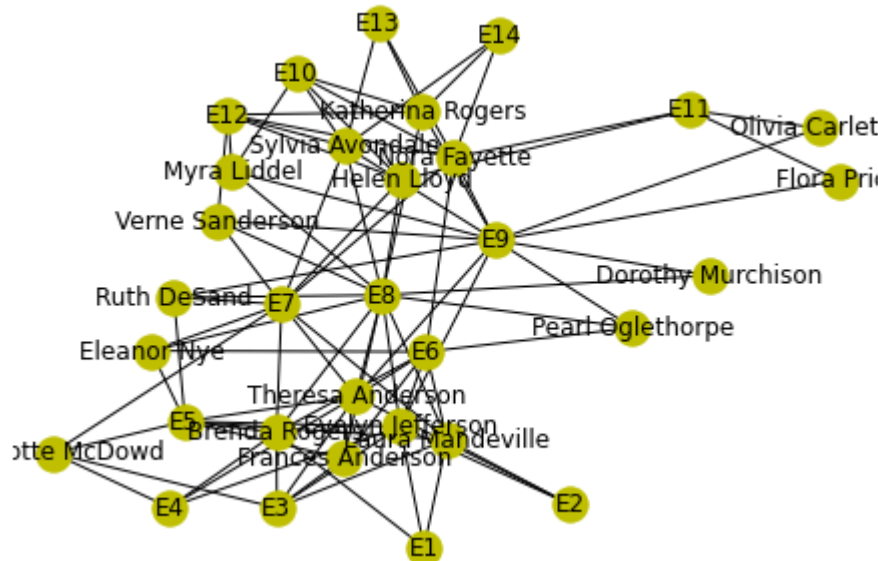
NAMES OF PARTICIPANTS OF GROUP I	CODE NUMBERS AND DATES OF SOCIAL EVENTS REPORTED IN <i>Old City Herald</i>													
	(1) 6/27	(2) 3/2	(3) 4/12	(4) 9/26	(5) 2/25	(6) 5/19	(7) 3/15	(8) 9/16	(9) 4/8	(10) 6/10	(11) 2/23	(12) 4/7	(13) 11/21	(14) 8/3
1. Mrs. Evelyn Jefferson.....	X	X	X	X	X	X	X	X	X					
2. Miss Laura Mandeville.....	X	X	X	X	X	X	X	X	X					
3. Miss Theresa Anderson.....	X	X	X	X	X	X	X	X	X					
4. Miss Brenda Rogers.....	X		X	X	X	X	X	X						
5. Miss Charlotte McDowd.....			X	X	X	X	X							
6. Miss Frances Anderson.....			X		X	X		X						
7. Miss Eleanor Nye.....				X	X	X	X	X						
8. Miss Pearl Oglethorpe.....					X	X	X	X	X					
9. Miss Ruth DeSand.....					X		X	X	X					
10. Miss Verne Sanderson.....							X	X	X			X		
11. Miss Myra Liddell.....								X	X	X		X		
12. Miss Katherine Rogers.....								X	X	X		X	X	X
13. Mrs. Sylvia Avondale.....							X	X	X	X		X	X	X
14. Mrs. Nora Fayette.....						X	X	X	X	X	X	X	X	X
15. Mrs. Helen Lloyd.....							X	X		X	X	X		
16. Mrs. Dorothy Murchison.....								X	X					
17. Mrs. Olivia Carleton.....								X			X			
18. Mrs. Flora Price.....								X			X			

The resulting bipartite graph on the vertex set consisting of the 18 woman and the 14 events is readily available in NetworkX .

```
In [15]: G = nx.davis_southern_women_graph()
print(G.nodes())
```

```
['Evelyn Jefferson', 'Laura Mandeville', 'Theresa Anderson', 'Brenda R
ogers', 'Charlotte McDowd', 'Frances Anderson', 'Eleanor Nye', 'Pearl
Oglethorpe', 'Ruth DeSand', 'Verne Sanderson', 'Myra Liddell', 'Katheri
na Rogers', 'Sylvia Avondale', 'Nora Fayette', 'Helen Lloyd', 'Dorothy
Murchison', 'Olivia Carleton', 'Flora Price', 'E1', 'E2', 'E3', 'E4',
'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13', 'E14']
```

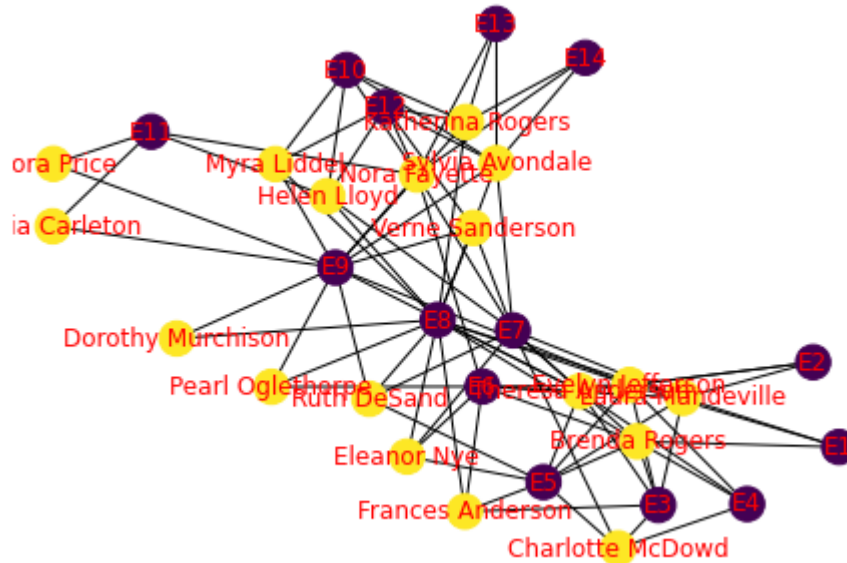
```
In [16]: nx.draw(G, **opts)
```




```
In [17]: color = nx.bipartite.color(G)
color = [color[x] for x in G.nodes()]
print(color)

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
In [18]: opts2["node_color"] = color
nx.draw(G, **opts2)
```



Note. The adjacency matrix A of a bipartite graph G , with respect to a suitable ordering of the vertices (B first, then W), has the form of a 2×2 -block matrix,

$$A = \begin{pmatrix} 0 & C \\ C^T & 0 \end{pmatrix}$$

where the blocks on the diagonal consist entirely of zeros, as there are no edges between vertices of the same color, and the lower left block is the **transpose** C^T of the matrix C of entries in the upper right.

```
In [19]: A = nx.adjacency_matrix(G)
print(A.toarray())
```

```
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
```

Oh. You don't really see much this way ...

```
In [20]: with np.printoptions(threshold=9999):
          print(A.toarray())
```

```
[
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 1 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0]
[1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 1 1 1 0 1 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 1 1 1 0 1 0 1 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 1 0 0 0 0 1 1 1 1 1 1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
]
```

- In NetworkX , all parts of a graph can have **attributes**: the nodes, the edges, and the graph object itself. Graph object attributes of a graph `G` are stored in the field `G.graph` . By convention, the two underlying sets of a bipartite graph are stored here as attributes called `'top'` and `'bottom'` .

```
In [21]: X, Y = G.graph['top'], G.graph['bottom']
C = nx.bipartite.biadjacency_matrix(G, X, Y)
print(C.toarray())
```

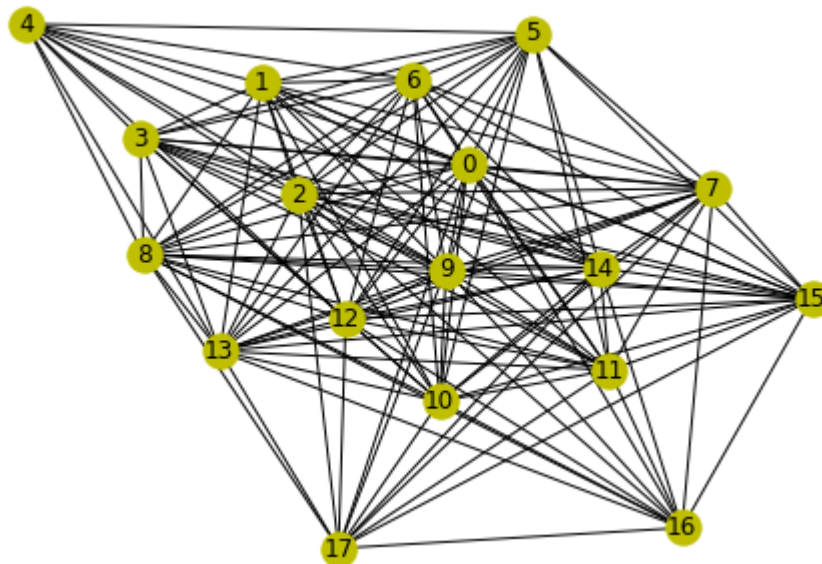
```
[[1 1 1 1 1 1 0 1 1 0 0 0 0 0 0]
 [1 1 1 0 1 1 1 1 0 0 0 0 0 0 0]
 [0 1 1 1 1 1 1 1 1 0 0 0 0 0 0]
 [1 0 1 1 1 1 1 1 0 0 0 0 0 0 0]
 [0 0 1 1 1 0 1 0 0 0 0 0 0 0 0]
 [0 0 1 0 1 1 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 1 1 1 1 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 1 1 0 0 0 0 0 0]
 [0 0 0 0 1 0 1 1 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 1 1 1 0 0 1 0 0 0]
 [0 0 0 0 0 0 0 1 1 1 0 1 0 0 0]
 [0 0 0 0 0 0 0 1 1 1 0 1 1 1]
 [0 0 0 0 0 0 1 1 1 1 0 1 1 1]
 [0 0 0 0 0 1 1 0 1 1 1 1 1 1]
 [0 0 0 0 0 0 1 1 0 1 1 1 0 0]
 [0 0 0 0 0 0 0 1 1 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 0 1 0 0 0]
 [0 0 0 0 0 0 0 0 1 0 1 0 0 0]]
```

As $A = A^T$, we get

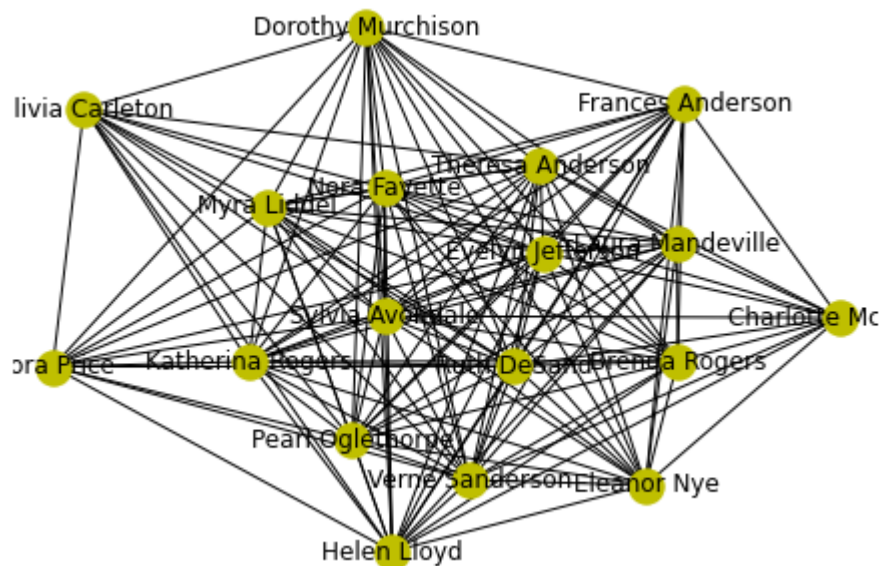
$$A^T \cdot A = A \cdot A^T = A \cdot A = \begin{pmatrix} C \cdot C^T & 0 \\ 0 & C^T \cdot C \end{pmatrix}$$

where $C \cdot C^T$ is the adjacency matrix of the **projection** onto the vertex set B , and $C^T \cdot C$ is the adjacency matrix of the **projection** onto the vertex set W .

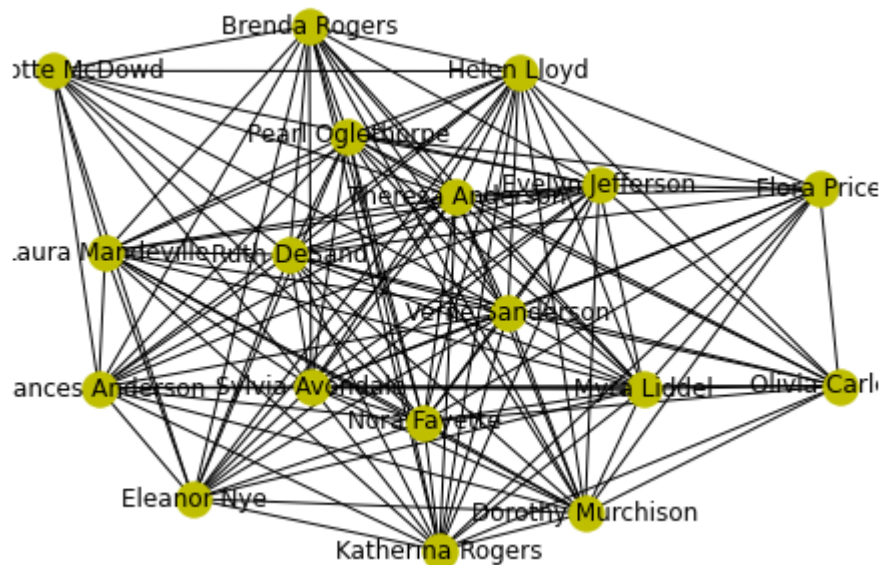
```
In [22]: BB = nx.from_numpy_matrix((C*C.transpose()).toarray())
nx.draw(BB, **opts)
```



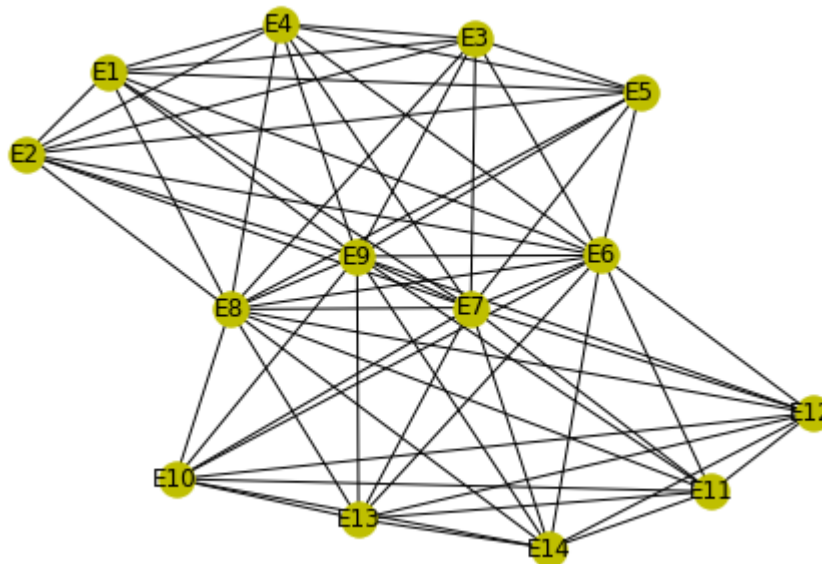
```
In [23]: nodes = G.graph['top']
mapping = {i: nodes[i] for i in range(len(nodes))}
nx.relabel_nodes(BB, mapping, False)
nx.draw(BB, **opts)
```



```
In [24]: BBB = nx.projected_graph(G, G.graph['top'])
nx.draw(BBB, **opts)
```



```
In [25]: WWW = nx.projected_graph(G, G.graph['bottom'])
          nx.draw(WWW, **opts)
```



```
In [26]: print((C*C.transpose()).toarray())
```

```
[[8 6 7 6 3 4 3 3 3 2 2 2 2 2 1 2 1 1]
 [6 7 6 6 3 4 4 2 3 2 1 1 2 2 2 1 0 0]
 [7 6 8 6 4 4 4 3 4 3 2 2 3 3 2 2 1 1]
 [6 6 6 7 4 4 4 2 3 2 1 1 2 2 2 1 0 0]
 [3 3 4 4 4 2 2 0 2 1 0 0 1 1 1 0 0 0]
 [4 4 4 4 2 4 3 2 2 1 1 1 1 1 1 1 0 0]
 [3 4 4 4 2 3 4 2 3 2 1 1 2 2 2 1 0 0]
 [3 2 3 2 0 2 2 3 2 2 2 2 2 2 1 2 1 1]
 [3 3 4 3 2 2 3 2 4 3 2 2 3 2 2 2 1 1]
 [2 2 3 2 1 1 2 2 3 4 3 3 4 3 3 2 1 1]
 [2 1 2 1 0 1 1 2 2 3 4 4 4 3 3 2 1 1]
 [2 1 2 1 0 1 1 2 2 3 4 6 6 5 3 2 1 1]
 [2 2 3 2 1 1 2 2 3 4 4 6 7 6 4 2 1 1]
 [2 2 3 2 1 1 2 2 2 3 3 5 6 8 4 1 2 2]
 [1 2 2 2 1 1 2 1 2 3 3 3 4 4 5 1 1 1]
 [2 1 2 1 0 1 1 2 2 2 2 2 2 2 1 1 2 1]
 [1 0 1 0 0 0 0 1 1 1 1 1 1 1 2 1 1 2]
 [1 0 1 0 0 0 0 1 1 1 1 1 1 1 2 1 1 2]]
```

Code Corner

python

- `divmod` : [\[doc\]](https://docs.python.org/3/library/functions.html#divmod) (<https://docs.python.org/3/library/functions.html#divmod>) the built-in quotient-and-remainder

```
In [27]: divmod(-7, 5)
```

```
Out[27]: (-2, 3)
```

numpy

- `printoptions` : [\[doc\]](https://numpy.org/doc/stable/reference/generated/numpy.printoptions.html) (<https://numpy.org/doc/stable/reference/generated/numpy.printoptions.html>) set options for printing arrays

networkx

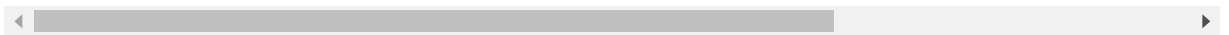
- `grid_2d_graph` : [\[doc\]](https://networkx.org/documentation/stable/reference/generated/networkx.generators.lattice.grid_2d_graph.html) (https://networkx.org/documentation/stable/reference/generated/networkx.generators.lattice.grid_2d_graph.html) creates a 2D grid graph.



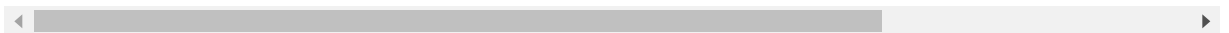
- `bipartite.color` : [\[doc\]](https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.color.html) (<https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.color.html>) computes a 2-coloring of a graph



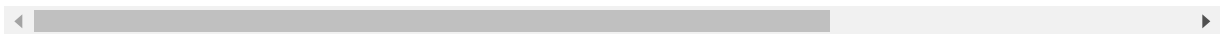
- `bipartite.biadjacency_matrix` : [\[doc\]](https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.biadjacency_matrix.html) (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.biadjacency_matrix.html) the incidence matrix of a bipartite graph



- `davis_southern_women_graph` : [\[doc\]](https://networkx.org/documentation/stable/reference/generated/networkx.generators.social.davis_southern_women_graph.html) (https://networkx.org/documentation/stable/reference/generated/networkx.generators.social.davis_southern_women_graph.html) the southern women network



- `projected_graph` : [\[doc\]](https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.projection.projected_graph.html) (https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.projection.projected_graph.html) the projected graph



Exercises

1. Compute the adjacency matrix of the bipartite graph B at the top of this page and verify its block structure.
2. Compute the biadjacency matrix C of the graph B .
3. Compute the two products of C and its transpose, and, using the products as adjacency matrix, construct two graphs from them.
4. Compute the two projections of the bipartite graph B and compare them with the graphs constructed in the previous exercise.

In []: